

Final project design 2

第一組

314512014 黃啟哲
314605004 陳昌駿
314512057 莫喬凱
313511056 蔡明晏

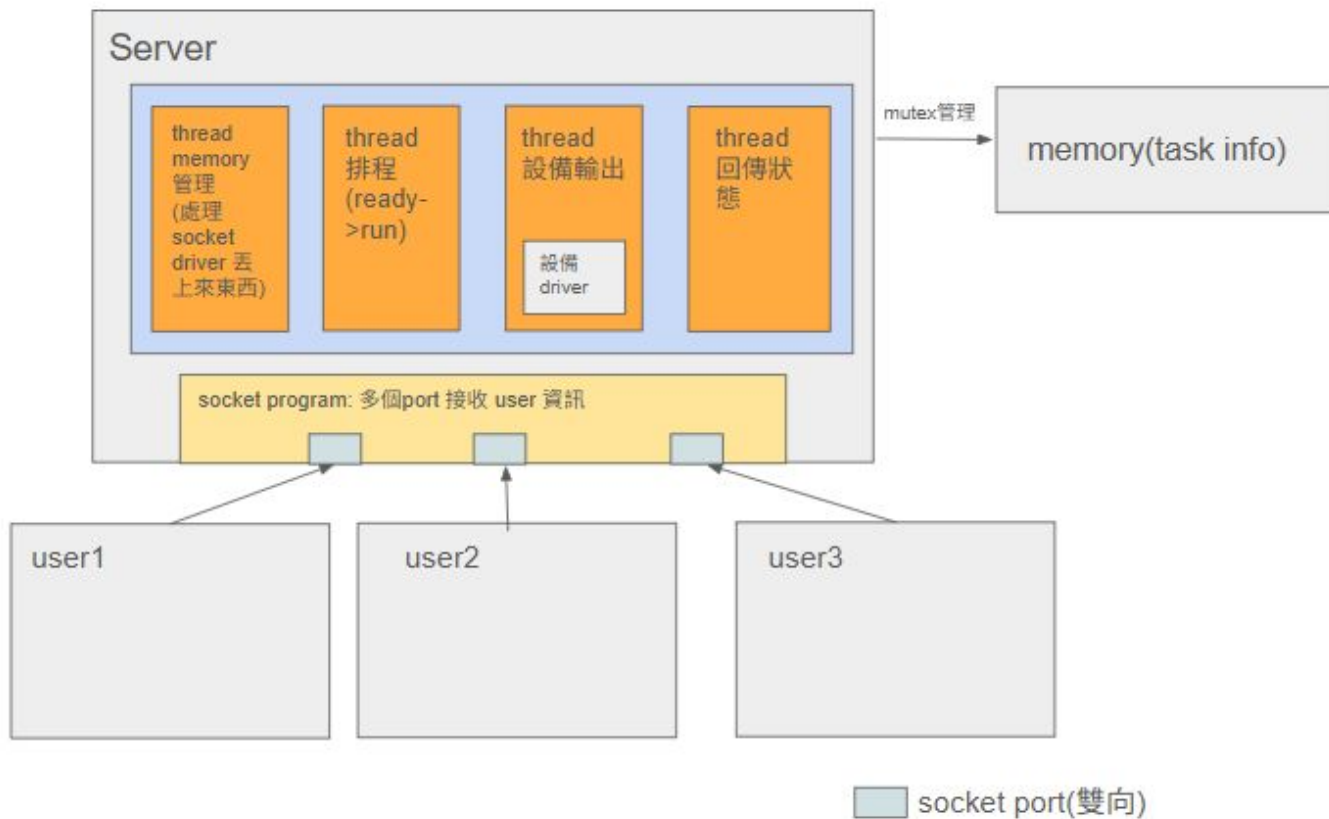


智慧家電預約系統

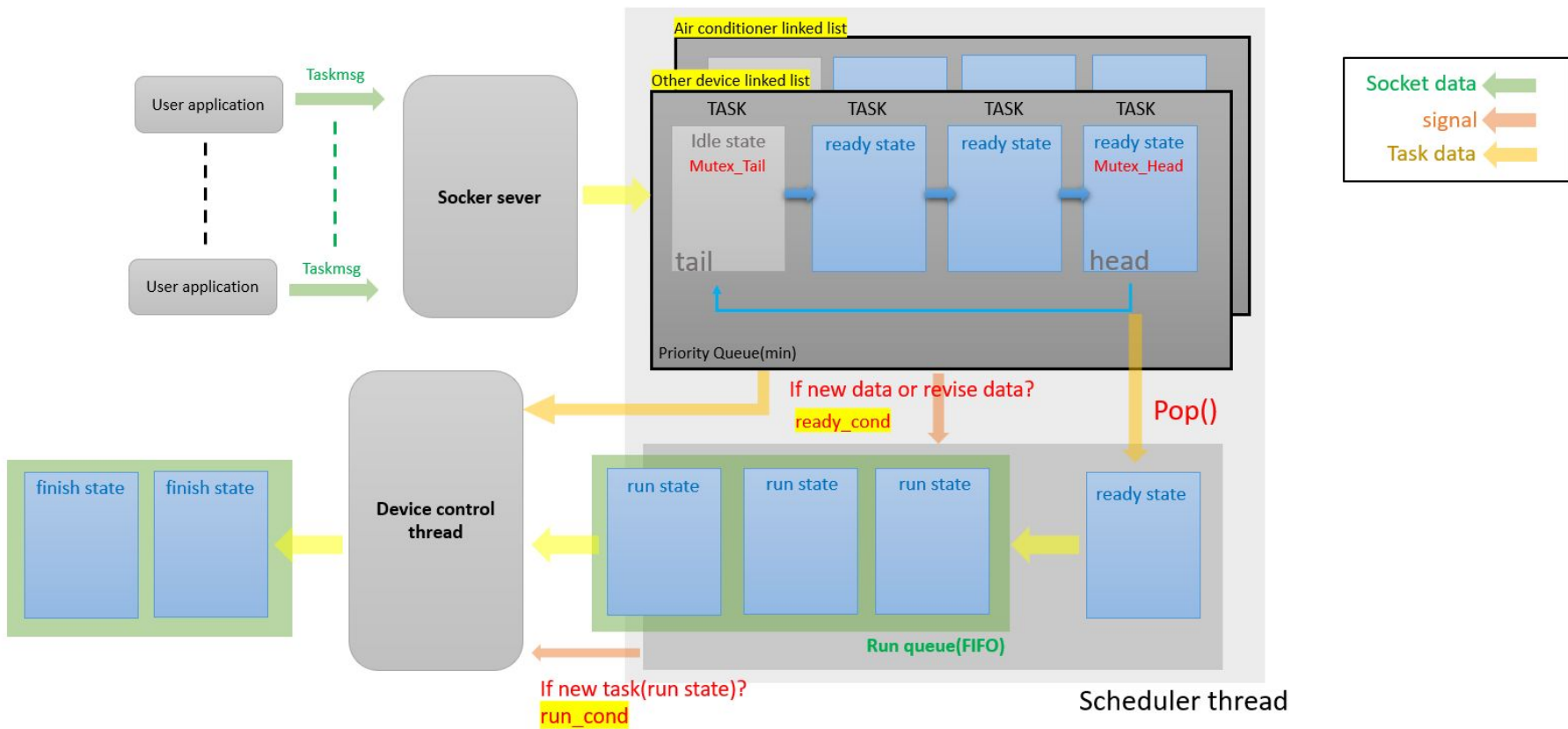


- 現在因為物聯網的發展越來越多智慧家居系列出現，讓使用者不必事事操心，只要預約好和設定一次即可讓智慧家電自行偵測情況並自行運作。
- 尤其宿舍是多人共用，學生在宿舍的時間又不固定，很適合實施智慧家居，可以省時省力，緩解器材尖峰時間的使用次數。

整體架構



整體架構-Data flow



Data Structure Definitions

```
typedef struct {
    int user_id;
    int room_id;
    int original_arrival_time;//17:8
    TaskMode task_mode;
} TaskMsg;

typedef enum{
    TASK_IDLE =0, // AFTER FINIISH SCHE SEI idle or 空閒
    TASK_READY, //heap finish
    TASK_RUN, //scheduler ready
    TASK_FINISH //all finish
}TaskState;

typedef struct
{
    bool MODE_WASH; // 洗
    bool MODE_DRY; // 烘
    bool MODE_AC; //ac
}TaskMode;
```

```
typedef struct {

    //task information=====
    TaskState task_state;
    TaskMode mode;// 冷氣(bool)
    int remain_time;//剩餘時間

    //user sending information=====
    int user_id;
    int room_id;
    int arrival_time;
    int original_arrival_time;
    int duration;          // 總耗時
    int start_threshold;// 最晚開始時間

    //schedule information(暫定)=====
    int order; //schedule order
    int is_sch; // 1= already sch, 0= not sch

    struct Task *next;
} Task;
```

```
typedef struct
{
    int ac[4]={0,0,0,0};
    int dry_m[1] ={0};
    int wash_m[1] ={0};
}device_state;
```

Application

登入or註冊:

可選擇三種devices做控制

將資訊存放在資料庫中

登入時輸入user_id 即可知道資訊

```
Is this your first time using the system?
1. Yes (sign up)
2. No (Load user data)
0. Exit Program
Please select: 314512057
Invalid input. Please select again.
Is this your first time using the system?
1. Yes (sign up)
2. No (Load user data)
0. Exit Program
Please select: 1

--- Sign up ---

1. Please enter your User ID: 314512057
2. Please enter your Room ID: 301
3. Please select devices to control :
1. Washer
2. Dryer
3. Air Conditioner
Enter device numbers separated by space or comma (e.g., 1, 3): 1 2

--- Selected devices ---
Controlled: Washer
Controlled: Dryer
--- User ID: 314512057, Room ID: 301 ---
```

```
Please select: 2
Please enter your User ID for verification: 314512057
User data found! User ID: 314512057, Room ID: 301.
```


Application

進入menu:

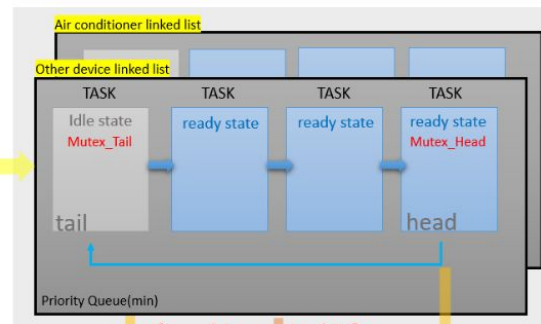
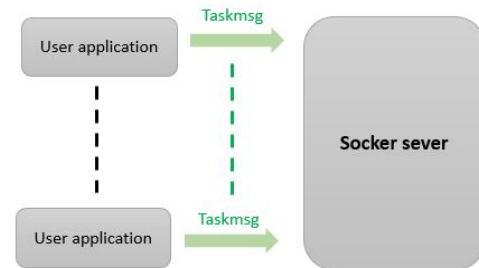
- 1.查 device status
- 2.設定預計到家時間
- 3.傳送資料給server
- 0.exit

```
=====
Smart Dormitory System Main Menu
  [Current Time: 21:57]
    User: 314512057 | Room: 301
=====
1. Display Device Status
2. Set Estimated Time of Arrival
3. Send Task Data to Server
0. Exit Main Menu and return to Startup Interface
Please select a function (0-3): █
```

```
Setting Estimated Time of Arrival
Enter estimated arrival time (e.g., 17:30): 23:00
Enter device numbers to schedule activation (e.g., 1 2): 1
--- Scheduled activation at 23:00 ---
Washer scheduled, Target Time: 23:00.
```

```
Device Status displaying (User ID: 314512057, Room ID: 301)
-----
Washer           Status: ON      Target Time: 23:00 (Remaining: 38 minutes)
Dryer            Status: OFF     Target Time: Not Scheduled
-----
```

Server(socket)



```

static void handle_client_task_msg(const TaskMsg *msg)
{
    bool flag_ac=false;
    bool flag_wh_dr=false;
    Task *t_device_wh_dr = malloc(sizeof(Task));
    if (!t_device_wh_dr) {
        perror("malloc Task");
        return;
    }
    memset(t_device_wh_dr, 0, sizeof(Task));

    Task *t_ac = malloc(sizeof(Task));
    if (!t_ac) {
        perror("malloc Task");
        return;
    }
    memset(t_ac, 0, sizeof(Task));

    //wh dry linked list task=====
    if(msg->task_mode.MOD_WASH || msg->task_mode.MOD_DRY){
        t_device_wh_dr->user_id = msg->user_id;
        t_device_wh_dr->room_id = msg->room_id;
        t_device_wh_dr->original_arrival_time = msg->original_arrival_time;
        t_device_wh_dr->task_state = TASK_IDLE;
        t_device_wh_dr->mode = msg->task_mode;
        t_device_wh_dr->mode.MODE_AC = false;

        t_device_wh_dr->is_sch = 0;
        t_device_wh_dr->order = 0;
        flag_wh_dr=true;
    }
    //=====

    //ac linked list task=====
    if(msg->task_mode.MOD_AC){
        t_ac->user_id = msg->user_id;
        t_ac->room_id = msg->room_id;
        t_ac->original_arrival_time = msg->original_arrival_time;
        t_ac->task_state = TASK_IDLE;
        t_ac->mode.MODE_AC = true;
        t_ac->mode.MODE_WASH = false;
        t_ac->mode.MODE_DRY = false;

        t_ac->is_sch = 0;
        t_ac->order = 0;
        flag_ac=true;
    }
    //=====
}
  
```

```

//Priority Queue operate=====
pthread_mutex_lock(&memlock);
if(flag_ac){
    handle_task_update_or_insert(&ready_ac_queue, t_ac);
    flag_ac=false;
}

if(flag_wh_dr){
    handle_task_update_or_insert(&ready_wh_dr_queue, t_device_wh_dr);
    flag_wh_dr=false;
}

pthread_mutex_unlock(&memlock);
  
```

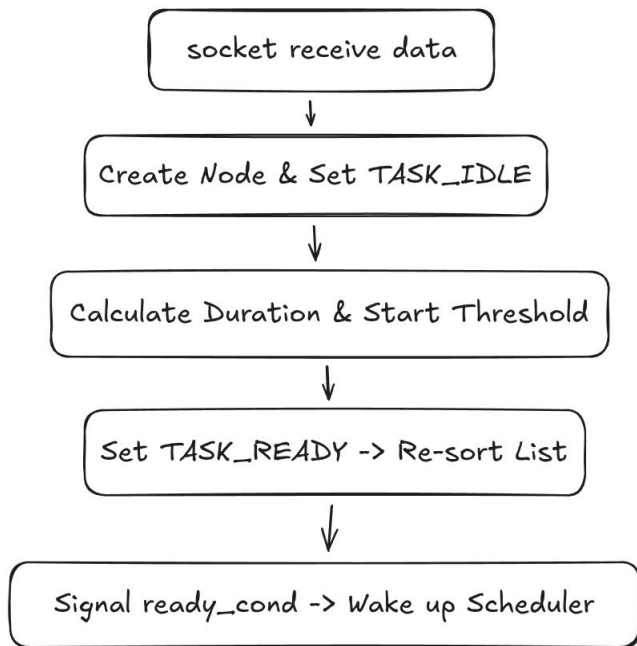

Scheduling

用linked list cycle管理task 排序

根據計算出的task啟動時間做排序

mutex鎖住頭尾task

冷氣task 獨立link list

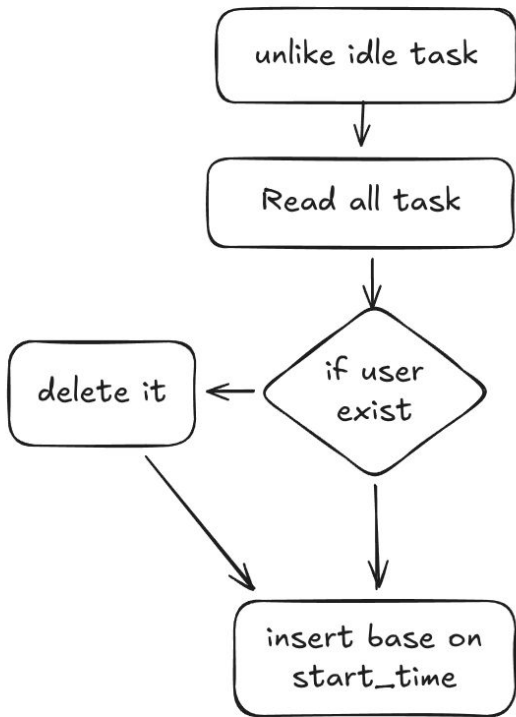


Scheduling-Priority Queue

排序邏輯使用Priority Queue

實作使用link-list, 避免長array的copy 浪費

相同user檢測



Scheduling-Priority Queue

```
do {
    next_node = get_next(current);

    if (current->task_state == TASK_IDLE) {
        printf("[MemMgr] Detected New Task for User %d.\n", current->user_id);

        // 1. clean up old tasks for the same user
        delete_old_tasks_for_user(current->user_id, current);

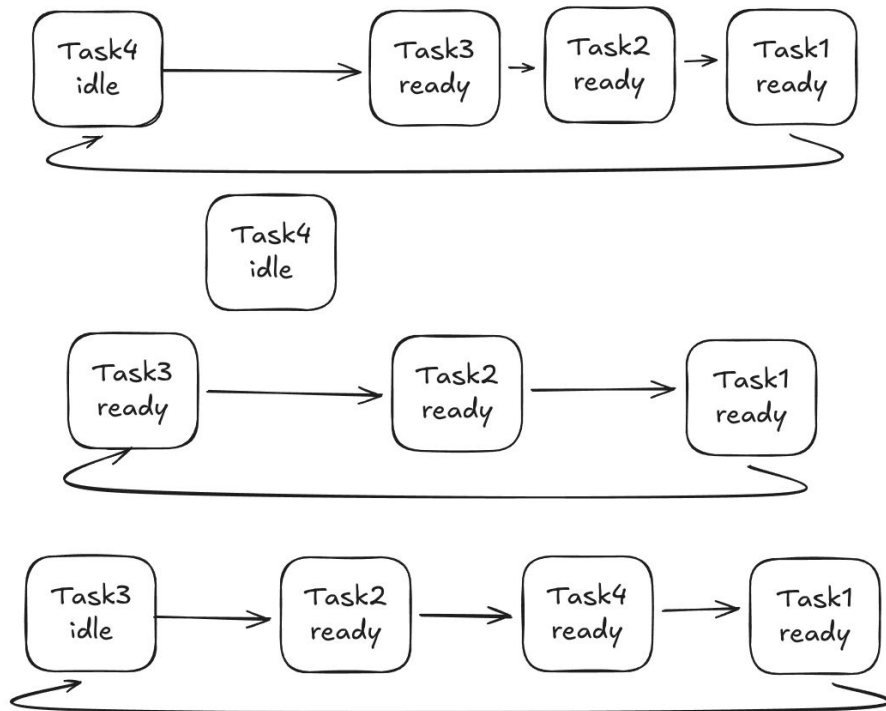
        // 2. calculate attributes (logic update)
        int now = get_current_time();
        current->duration = calculate_duration(current->mode);
        calculate_task_arrive_and_threshold(now, current);

        // 3. Re-sort based on the new start_threshold
        Task* node_to_move = unlink_node(&g_task_head, current);
        if (node_to_move) {
            insert_task_sorted(&g_task_head, node_to_move);
        }

        // 4. change state to READY
        current->task_state = TASK_READY;

        // 5. send signal to Scheduler
        printf("[MemMgr] Task Ready. Signaling Scheduler.\n");
        pthread_cond_signal(&ready_cond);
        break;
    }

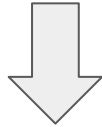
    current = next_node;
    if (current == g_task_head) wrapped = true;
} while (!wrapped && g_task_head != NULL);
```



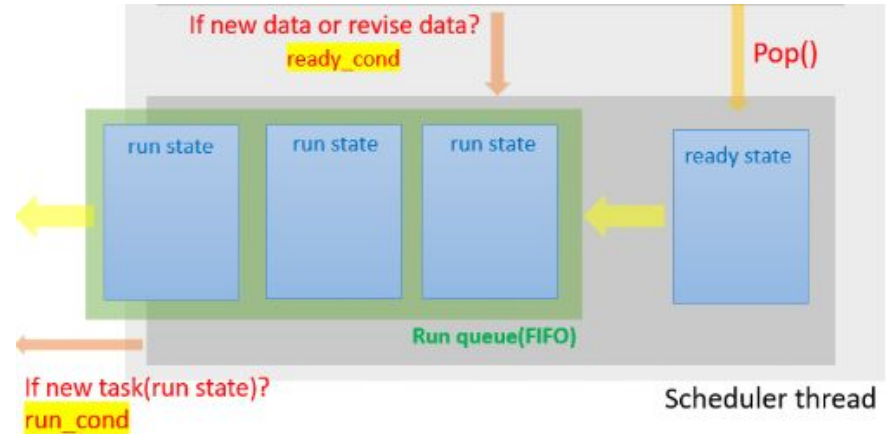
Scheduling-Pop The Priority Queue $\Rightarrow$ Running Queue

```
t = pop_min_heap(&ready_heap);  
if(!t){  
    break;  
}
```

```
//(1)wait,if ready queue no data ==> sleep in ready_cond  
while(ready_heap.size==0){  
    pthread_cond_wait(&ready_cond,&memlock);  
}
```



```
//deal with different device  
queue_push(&run_queue,t);  
pthread_mutex_unlock(&memlock);  
pthread_cond_signal(&run_cond);
```



Device

run_queue排序完成後會發送signal通知driver可以讀取

在讀取run_queue的資訊後，根據TaskMsg啟用特定機器

洗衣機: 7-segment + 3 燈泡顯示 state, 完成後啟動烘衣機

烘衣機: 4顆燈泡倒數計時

冷氣: 根據房間控制各自的冷氣

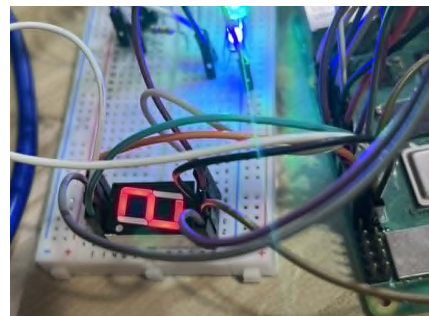
完成後將任務完成訊息通知給user

```
typedef struct {  
    int user_id;  
    int room_id;  
    int original_arrival_time; // 17:8  
    TaskMode task_mode;  
} TaskMsg;  
  
typedef struct  
{  
    bool MODE_WASH; // 洗  
    bool MODE_DRY; // 烘  
    bool MODE_AC; // ac  
} TaskMode;
```

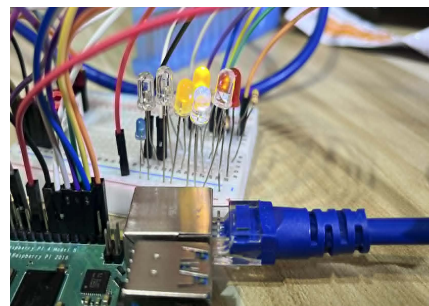
Device

```
int main() {  
  
    if (display_init() < 0) {  
        printf("Failed to init display\n");  
        return -1;  
    }  
  
    show_washing_machine_state();  
    show_dryer_state();  
    show_AC_state();  
  
    return 0;  
}
```

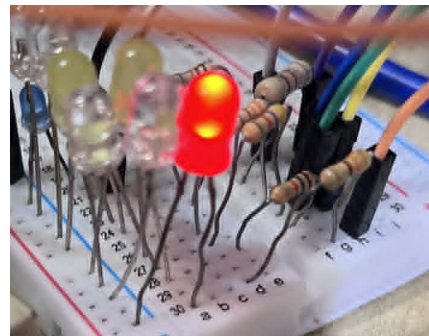
洗衣機



烘衣機



冷氣



工作分配

sever(socket) 、 scheduler : 黃啟哲

memory 管理、Priority Queue : 陳昌駿

application、多user設計: 莫喬凱

device、整合: 蔡明晏

謝謝大家